Routes

Learning Objectives

- Explain that client-side routing is the ability for a JavaScript application to map different URLs to their pages and render them in the browser
- Explain that React Router is an implementation of client-side routing
- Explain the difference between a URL and a path
- Build a React application that uses React Router and has two different URLs a user can navigate to and see different content without the page reloading
- Build a Route component with a dynamic path
- Explain that the value of a dynamic path is passed as a parameter, and can be accessed using the React Router useParams hook
- Build a React application that has Routes, Route, and Link components, and uses the useParams hook to access data

Intro

The apps we build get **bundled** within the same HTML file - we let JS/React handle all the logic of what to show and when to show it, e.g. by clicking on buttons in the UI we present. This type of approach is commonly called **S**ingle **P**age **A**pplication (**SPA**)

Q. But what happens if we try to navigate to a different URL using the browser's URL bar?

Example Single Page App

Let’s check this out using this app

Intro

We can’t use the address bar to navigate to any part of our app other than the main page! Fear not, because we have **JavaScript** and a **browser event** to our rescue!

Intro

In Vanilla JS, the History API is one way to do this:

https://developer.mozilla.org/en-US/docs/Web/API/History

React Router

React Router makes the History API nice and easy to use in React!

React Router

With React Router, for each **URL we choose**, we can render **specific components** of our app.

This way, users can use the browser's URL bar to navigate to different pages of an application!

React Router

As with all external packages, we start by **installing React Router** as a dependency of our project by using **NPM**

```
$ npm install react-router-dom@6 --save
```

RESOURCES: React Router 6 Documentation

Warning! If you search for React Router on Google, the v5 documentation will show up first. V5 has a different syntax so be sure to use the link above.

React Router

Then, we can **import** it into our project through a regular import statement

```
import { BrowserRouter } from "react-router-dom";
```

React Router

Next step will be to make the **router** available throughout our app

For this, we're going to wrap our App component with the **BrowserRouter** component we've just **imported**

This way, all the browser URL data will be passed down into our app via props!

```
import { BrowserRouter } from "react-router-dom";

root.render(
 <React.StrictMode>
   <BrowserRouter>
     <App />
   </BrowserRouter>
 </React.StrictMode>,
 rootElement
);
```

React Router

Now we're ready to split our app

First, we import a **Routes** component

This **ensures** that we only render **one** part of our app **at a time**, depending on the URL

```
import { Routes, Route } from "react-router-dom";

<Routes>
  <Route
    path="/products"
    element={<ProductsPage products={products} />}
  />
</Routes>
```

React Router

We also need to import the **Route** component.

This component will check if the **browser URL** path **matches** the string that we give it in the **path** prop

If it does, then the **Routes** component will **render** the component inside the `element` attribute

```
import { Routes, Route } from "react-router-dom";

<Routes>
 <Route
   path="/products"
   element={<ProductsPage products={products} />}
 />
</Routes>;
```

React Router

In order to navigate to our routes we can use **Link**, which is a wrapper around the HTML `<a>` element.

```
import { Routes, Route, Link } from "react-router-dom";

<Link to="/products">Products</Link>
```

Time for some live coding

DEMO 1

Create navigation with the following routes:

- **“/products” for the Products Page**
- **“/” for the Home Page**

Time for some practice

https://github.com/boolean-uk/react-routes-practice-1

React Router

```
{products.map((product, index) => {
  return (
    <li key={index}>
      <h3>{product.name}</h3>
      <p>£{product.price}</p>
      <Link to={`/products/${product.id}`}>
        View Product
      </Link>
    </li>
  );
})}
```

The next page we will create is to view a single product located at `"/products/1"`

To do this, we set the the `to` attribute of **Link** component to dynamically generate the path location based on the product `id` attribute

React Router

```
<Routes>
  <Route
    path="/products/:id"
    element={<ViewProductPage products={products} />}
  />
 ...
</Routes>
```

Use a `Route` to render the right component where the `path` uses a parameter (`:id`) so that React tries to matches the url based on this pattern e.g. `localhost:5173/products/123`

We also need to pass the `products` state down as a prop to the component

React Router

```
import { useParams} from "react-router-dom";

function ViewProductPage(props) {
  const [product, setProduct] = useState(null);
  const { products } = props;
  const { id } = useParams();

  useEffect(() => {
    if (products && id) {
      const matchingProduct = products.find((product) =>
Number(product.id) === Number(id));
      setProduct(matchingProduct);
    }
  }, [products, id]);
  ...
}
```

The **useParams** hook returns an object of key/value pairs that were matched by the `<Route path>`

In the previous slide, we defined a parameterised path with a parameter of `:id` - meaning the key will be `id` (no colon) when we call **useParams()**.

We use object destructuring to create a variable called `id` which will have a value based on the actual path we are visiting. So, if we visit …/products/22, then the `id` variable here will have a value of 22.

We passed the `products` data down as props. So, now, we are able to use the Array `.find` method to find the product which matches the `id` in the path parameter.

Note the variables that we have in our `useEffect` dependency array since we are referencing/using external data that can change.

Time for some live coding

Demo 2

Demonstrate passing data with the Link:

- **Add a Link component to the products page**
- **Create the “/products/:id” route**
- **Use the “useParams” hook to filter data**

useNavigate

The **useNavigate** hook returns a function that you can use to navigate programmatically.

https://reactrouter.com/en/main/hooks/use-navigate

useNavigate

First, **import** it into a project through a regular import statement

```
import { useNavigate } from "react-router-dom";
```

useNavigate

Then, assign the function to a variable

```
const navigate = useNavigate();
```

useNavigate

Now you can navigate programmatically by:

- Passing a string value to navigate to a specific path
- Passing a positive/negative integer to jump forwards/backwards based on your browser tab history

```
<button onClick={() => navigate("/")}>
  Go to Home Page
</button>

<button onClick={() => navigate(1)}>
  Go Forward
</button>

<button onClick={() => navigate(-1)}>
  Go Back
</button>

<button onClick={() => navigate(-2)}>
  Go Back 2 Pages
</button>
```

Time for some live coding

Demo 3

Implement useNavigate to:

- **Add buttons to the ViewProduct pages to “Go Home”, “Go Back” and “Go Forward”**

Time for some practice

https://github.com/boolean-uk/react-routes-practice-2